SOFTWARE TESTING, VERIFICATION AND RELIABILITY
Softw. Test. Verif. Reliab. 2016; **26**:351–365
Published online 7 March 2016 in Wiley Online Library (wileyonlinelibrary.com). DOI: 10.1002/stvr.1600

A general modeling and analysis framework for software fault detection and correction process

Yu Liu[1], Duo Li[1,*,†], Lujia Wang[2] and Qingpei Hu[2]

[1]*Key Laboratory of Advanced Reactor Engineering and Safety, Institute of Nuclear and New Energy Technology, Tsinghua University*
[2]*Academy of Mathematics and Systems Science, Chinese Academy of Sciences*

SUMMARY

Software reliability growth modeling plays an important role in software reliability evaluation. To incorporate more information and provide more accurate analysis, modeling software fault detection and correction processes has attracted widespread research attention recently. In modeling software correction processes, the assumption of fault correction time is relaxed from constant delay to random delay. However, stochastic distribution of fault correction time brings more difficulties in modeling and corresponding parameter estimation. In this paper, a framework of software reliability models containing both information from software fault detection process and correction process is studied. Different from previous extensions on software reliability growth modeling, the proposed approach is based on Markov model other than a nonhomogeneous Poisson process model. Also, parameter estimation is carried out with weighted least-square estimation method, which emphasizes the influence of later data on the prediction. Two data sets from practical software development projects are applied with the proposed framework, which shows satisfactory performance with the results. Copyright © 2016 John Wiley & Sons, Ltd.

Received 25 August 2014; Revised 21 November 2015; Accepted 17 January 2016

KEY WORDS: fault detection process; fault correction process; software reliability; weighted least-square estimation

Notations

N	number of faults in the software
N'	number of time intervals in fault detection process
M'	number of time intervals in fault correction process
$m_d(t)$	mean value function of fault detection process
$m_c(t)$	mean value function of fault correction process
$\lambda_d(t)$	the intensity function of fault detection process
$f_c(t)$	the probability density function of correction time
n_i	the cumulative number of faults detected by time
m_i	the cumulative number of faults corrected by time
θ	the expected total number of faults
λ_i	detection intensity of the ith fault
γ_i	correction intensity of the ith detected fault
$Var[N_d(t)]$	the variance of expected number of detected faults by time t
$Var[N_c(t)]$	the variance of expected number of corrected faults at time t

*Correspondence to: Duo Li, Key Laboratory of Advanced Reactor Engineering and Safety, Institute of Nuclear and New Energy Technology, Tsinghua University.
†E-mail: liduo@tsinghua.edu.cn

Copyright © 2016 John Wiley & Sons, Ltd.

1. INTRODUCTION

In the past few decades, the software reliability growth models (SRGMs) have attracted widespread research attention and are applied in the software development process [1, 2]. Focusing on the software development life cycles [3] and via black box testing method, most SRGMs utilize the fault detected data during testing process to describe the stochastic behavior of the software fault detection process (FDP) with respect to time, in which the most popular parametric models are the non-homogeneous Poisson process (NHPP) models [4–7] and Markov models [2, 8–10]. With more and more SRGMs being proposed and studied, a unification research is directed to establish a unified modeling framework composed of typical reliability growth patterns. Some unification schemes are proposed, such as a Generalized Order Statistic (GOS) model [11] a semi-parametric model, which is put forward by Ross [12], Kapur's generalized imperfect debugging NHPP models [13], and a unified modeling framework of Markov-type software reliability models [14]. Similar to the GOS models, a novel model is developed, which ideally assumes each fault in the software independently causing errors to occur with exponential order statistic process whose error intensity is unknown [15].

However, most of these modeling frameworks are under the idealized assumptions that all the faults are removed instantaneously and perfectly and the expected number of removed faults is the same as the expected number of detected faults. In fact, it always needs time for fault removal, and the expected number of faults removed at any given time is less than the expected number of detected faults [16]. Since then, some researchers also took into account of the fault correction process (FCP) and used the fault corrected data to represent the correction time delay. Modeling both FDP and FCP takes more information from software testing records and improves the estimation and prediction results. For instance, *Schneidewind* proposed an approach for modeling the fault correction process by using a constant delayed fault-detection process [17]. He assumed that the rate of fault correction was proportional to the rate of fault detection. However, because the FCP depends heavily on the FDP, in some applications, the model will underestimate the remaining faults in the code. Later, Xie *et al*. extended the *Schneidewind* model to a continuous version by substituting a time-dependent delay function for the constant delay [18, 19]. Based on the proposed FDP and FCP modeling framework and NHPP assumption, both the least square estimation and maximum likelihood estimation approaches have been proposed in the research. Some researchers incorporated the FCP with a queueing theory, and assumed an infinite number of servers [20, 21]. Later, a neural networks configuration approach with an extra factor characterizing the dispersion of prediction repetitions is applied to model these two processes together [22]. Recently, incorporating testing effort function and imperfect debugging, Peng proposed a framework to analyze those processes [23]. On the other hand, extensions on the Markov models have also been conducted by Gokhale in a systematic way [16]. The Markov-model-based FDP and FCP modeling framework incorporates both the non-zero fault correction time and fault introduction factors. Analysis for this framework is conducted in a simulated way, which is applied for the decision-making of fault correction policies.

In this paper, a novel FDP and FCP modeling framework based on the GOS models is proposed, with both the modeling and statistical-analysis methods. Based on the traditional GOS models for FDP, the FCP is regarded as a delayed one. Apart from FDP and FCP modeling, the parameter estimation is also an important issue in research. It is well accepted that the ML estimation has many desirable properties, such as asymptotic-normality, admissibility, and consistency [24], and some complex likelihood functions for combined FDP & FCP are developed [18, 20]. However, ML has some inherent defects [25] and can only be applied to detection and correction data with an equal time interval. In the Jelinski–Moranda model, which is the earliest GOS model, the ML estimate does not necessarily exist, and it is not necessarily unique [26]. The ordinary least-square (OLS) estimation method is a more common method to estimate parameters due to its simplicity. While, the OLS estimate lacks a sound theoretical explanation and ignores the different variance of each data point [27]. To improve OLS estimate and try to balance the heteroscedasticity, which is introduced by the testing mechanism, the weighted least square (WLS) estimation is applied here.

The rest of this paper is organized as follows. Section 2 outlines the unification framework for FDP and FCP modeling. In section 3, the parameter estimation with WLS estimate is proposed and

Copyright © 2016 John Wiley & Sons, Ltd.

DOI: 10.1002/stvr

10991689, 2016, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/stvr.1600 by PUC/PR - Pontificia Universidade Catolica do Parana, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

developed. Section 4 illustrates the application of proposed FDP and FCP models with two practical data sets. And then, reliability analysis and optimal release time issues are addressed. The future research discussion is shown as a conclusion in section V.

2. MODELING FRAMEWORK

Assumptions:

1. Based on the debugging theory mentioned in [28], assume that the software faults occur at independent distributed random times, as a fact of the unknown location of fault.
2. The initial number of the faults N is positive and finite. This is a common assumption that has been used in most of software reliability growth model [5, 29], which can be easily relaxed by introducing more model parameters.
3. When a fault remains in the code, its state remains in 0. When the ith fault is detected, its state changes to 1 with detection rate λ_i, and when it is corrected, its state changes to 2 with correction rate μ_i.



The fault detection rate is related to the debuggers' detection capacity, the test case testability and test coverage. The fault correction rate is related to the debuggers' correction capacity and the complexity of each fault. In this assumption, this is reasonable to assume each detected fault will enter the correction process immediately, because during the test of large-scale software, the fault detection and correction are conducted in parallel.

Those general assumptions allow us to incorporate the fault correction data into modeling, and easily develop more specific models suitable in different situations. More parameters can be added for more flexible use.

2.1. Modeling Fault Detection Process

Let λ_1 denotes the error detection intensity of the ith fault, and $I_{di}(t)$ is the indicator function of the ith fault's state, which

$$I_{di}(t) = \begin{cases} 1, & \text{Fault has not been detected} \\ 0, & \text{Otherwise} \end{cases}.$$

Then the ith fault detected Probability Density Function (PDF) obeys, for example, exponential distribution and can be described as follows:

$$f_{di}(t) = \lambda_i e^{-\lambda_i t}. \tag{1}$$

Because the error rate of every fault is λ_i, the expected error rate of the software $\Lambda_d(t)$ is

$$E\left[\Lambda_d(t)\right] = \sum_{i=1}^{N} \lambda_i E\left[I_{di}(t)\right] = \sum_{i=1}^{N} \lambda_i \int_t^{\infty} f_{di}(\tau) d\tau = \sum_{i=1}^{N} \lambda_i e^{-\lambda_i t}. \tag{2}$$

Then, the mean value function (MVF) of fault detection number is formulated as follows,

$$m_d(t) = E(N_d(t)) = \sum_{i=1}^{N} [1 - \exp(-\lambda_i t)]. \tag{3}$$

Copyright © 2016 John Wiley & Sons, Ltd.

DOI: 10.1002/stvr

10991689, 2016, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/stvr.1600 by PUC/PR - Pontificia Universidade Catolica do Parana, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

The variance of the detected fault number by time t presented in Equation (3) can be derived as

$$\begin{aligned} Var(N_d(t)) &= \sum_{i=1}^{N} [Var\,(1 - I_{di}(t))] \\ &= \sum_{i=1}^{N} [Var\,(I_{di}(t))] = \sum_{i=1}^{N} \left[E(I_{di}(t)) - E^2(I_{di}(t))\right] \end{aligned} . \tag{4}$$

For mathematical purposes, the potential number of initial faults N is viewed as real value even though it is really integer valued. And if it assumes that $\lambda_i = \lambda_0, i = 1, 2, \ldots,$ [11], the MVF can be simplified to the famous Jelinski-Moranda (J-M) model [10]:

$$m_d\,(t) = E\,(N_d(t)) = N\,(1 - \exp\,(-\lambda_0 t))\,. \tag{5}$$

For the J–M model, the variance of the detected fault number by time t is as follows, which has been proposed before by Koh [15],

$$\begin{aligned} Var(N_d(t)) &= N(\exp(-\lambda_0 \cdot t) - \exp(-2\lambda_0 \cdot t)) \\ &= m_d(2t) - m_d(t). \end{aligned} \tag{6}$$

2.2. Modelling Fault Correction Process

To model FCP, assume that μ_i is the error correction intensity of the ith fault that has been detected, and $I_{ci}(t) = 0$ denotes the ith fault has been corrected prior to t and $I_{ci}(t) = 1$ if not, then

$$E(I_{ci}(t)) = P(t_d + \Delta t_c > t), \tag{7}$$

where t_d and Δt_c refer to the duration of fault detection and correction.

According to previous assumptions and using exponential distribution, which is applied in fitting random fault correction time in many practical software testing projects [30], the PDF of successfully correcting ith fault can be described as follows:

$$f_{\Delta ci}(\Delta t) = \mu_i \exp(-\mu_i \cdot \Delta t). \tag{8}$$

Because every fault will enter FCP immediately after being detected, then the error correction process can be regarded as an overlay of two GOS processes as follows:

$$\begin{aligned} E(I_{ci}(t)) &= \int_t^{\infty} \int_0^{\tau} (f_{di} * f_{\Delta ci})(\tau) d\tau = \int_t^{\infty} \int_0^{\tau} f_{di}(x) f_{\Delta ci}(\tau - x) dx d\tau \\ &= \begin{cases} \frac{\mu_i e^{-\lambda_i t} - \lambda_i e^{-\mu_i t}}{\mu_i - \lambda_i}, & \lambda_i \neq \mu_i \\ (\lambda_i t + 1) e^{-\lambda_i t}, & \lambda_i = \mu_i \end{cases}, \quad \lambda_i \mu_i > 0. \end{aligned} \tag{9}$$

Similar to the derivation of MVF in FDP, the MVF in FCP can be derived as

$$N_c(t) = \sum_{i=1}^{N} (1 - I_{ci}(t)), \tag{10}$$

$$E(N_c(t)) = \sum_{i=1}^{N} E(1 - I_{ci}(t)) = \sum_{i=1}^{N} (1 - E(I_{ci}(t))). \tag{11}$$

Copyright © 2016 John Wiley & Sons, Ltd.

DOI: 10.1002/stvr

10991689, 2016, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/stvr.1600 by PUC/PR - Pontificia Universidade Catolica do Parana, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

If combining with the J–M model and assuming $\mu_i = \mu_0, 1, 2, \ldots, N$, the MVF of FCP is formulated as follows:

$$m_c(t) = E\left[N_c(t)\right] = \begin{cases} N\left(1 - \frac{\mu_0 e^{-\lambda_0 t} - \lambda_0 e^{-\mu_0 t}}{\mu_0 - \lambda_0}\right) & , \lambda_0 \neq \mu_0 \\ N\left(1 - (\lambda_0 t + 1)e^{-\lambda_0 t}\right) & , \lambda_0 = \mu_0 \end{cases}, \lambda_0 \mu_0 > 0. \tag{12}$$

The previous formulation is similar to the FCP model proposed by Wu *et al*. [18], which follows NHPP. The most obvious difference between these two models is that, the GOS model supposes the number of initially contained faults is finite while the NHPP model assumes it as a Poisson distributed random variable with the mean value of N [31]. Aside from this, their variances of the expected cumulated number of faults have different tendency, which will be illustrated in following content.

With different assumptions on the distribution of correction time delay and the detection rate, the proposed model framework can have different formats. The different formats with the exponential detection rate are similar with the models proposed by Wu. *et al*. [18].

The variance of $N_c(t)$ is derived as

$$\begin{aligned} Var(N_c(t)) &= \sum_{i=1}^{N}\left[E\left(I_{ci}^2(t)\right) - E^2\left(I_{ci}(t)\right)\right] \\ &= N - \left[\sum_{i=1}^{N} E^2\left(I_{ci}(t)\right)\right] - m_c(t). \end{aligned} \tag{13}$$

When describing the FDP and FCP with the model shown in Equation (12), the Variance of $N_c(t)$ can be simplified as follows:

$$Var(N_c(t)) = N - \frac{(N - m_c(t))^2}{N} - m_c(t). \tag{14}$$

According to Equation (14), It can be easily proved that, the expected variance of $N_c(t)$ increases with testing time t when $m_c(t) < N/2$, and decreases when $m_c(t) \geq N/2$.

Above all, the proposed model is different from the existed model based on NHPP [18–21] for the following reasons:

1. Proposed model is based on General Order Statistic, and the initial fault count embedded in software is fixed, while the NHPP-based model is assumed the initial faults count is distributed as Poisson distribution.
2. The MVF of proposed model has a decreasing variance when t is big enough, and the MVF of NHPP based model always possesses an increasing variance with t, which is

$$Var_d^{(NHPP)}(t) = m_d(t), Var_c^{(NHPP)}(t) = m_c(t).$$

The proposed modeling framework allows researchers to incorporate the fault correction data into modeling, and easily generate various models with different assumptions on fault detection and correction distributions.

2.3. Parameter Estimation with Weighted Least-Square Estimation Method

OLS estimate is simple in calculation and widely used in SRGM, but it lacks a sound theoretical explanation and ignores the different variance of each data point [27], which is introduced by the testing mechanism. To improve OLS and try to balance the variance, the weighted least square estimation is used here.

Copyright © 2016 John Wiley & Sons, Ltd.

DOI: 10.1002/stvr

10991689, 2016, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/stvr.1600 by PUC/PR - Pontificia Universidade Catolica do Parana, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Suggested by Koh. [15], construct new non-heteroscedasticity variables independent with t,

$$\begin{cases} \frac{n_k - E\left(N_d\left(t_k^d\right)\right)}{\sqrt{Var\left(N_d\left(t_k^d\right)\right)}} = \varepsilon_k^d\left(t_k^d, \theta\right) \\ \frac{m_j - E\left(N_c\left(t_j^c\right)\right)}{\sqrt{Var\left(N_c\left(t_j^c\right)\right)}} = \varepsilon_j^c\left(t_j^c, \theta\right) \end{cases} \tag{15}$$

in which θ refers to the parameters to be estimated in the model, t_k^d (t_j^c) refers to the kth(jth) recording time of FDP (FCP). It is worth noting that in some software testing record, the recording time interval of FDP and FCP can be different from each other, that is, when $k = j$, not every t_k^d is equal to t_j^c.

It can be proved that the second moment of ε_d and ε_c equal to 1, use the logarithm of the normalized second central moment of $m_d(t)$ and $m_c(t)$, that is, $\ln(\varepsilon_2^d)$ and $\ln(\varepsilon_2^d)$ to construct the optimal weighted Least-square estimation, which can be written as

$$\begin{aligned} \min_{\theta} S^2 &= \sum_{k=1}^{N'} \left\{ \ln\left[\varepsilon_k^d\left(t_k^d, \theta\right)\right]^2 \right\}^2 + \sum_{j=1}^{M'} \left\{ \ln\left[\varepsilon_j^c\left(t_j^c, \theta\right)\right]^2 \right\}^2 \\ &= \sum_{k=1}^{N'} \left\{ 2\ln\left|m_d\left(t_k^d, \theta\right) - n_k\right| - \ln\left[Var\left(N_d\left(t_k^d, \theta\right)\right)\right] \right\}^2 \\ &\quad + \sum_{j=1}^{M'} \left\{ 2\ln\left|m_c\left(t_j^c, \theta\right) - m_j\right| - \ln\left[Var\left(N_c\left(t_j^c, \theta\right)\right)\right] \right\}^2 \\ &= \sum_{k=1}^{N'} \left[2\ln\left(\left|m_d\left(t_k^d\right) - n_k\right|\right) + 2\ln\left(w_d\left(t_k^d\right)\right) \right]^2 \\ &\quad + \sum_{j=1}^{M'} \left[2\ln\left(\left|m_c\left(t_j^c\right) - m_j\right|\right) + 2\ln\left(w_c\left(t_j^c\right)\right) \right]^2, \end{aligned} \tag{16}$$

in which

$$w_d(t) = \frac{1}{\sqrt{Var(N_d(t))}}, w_c(t) = \frac{1}{\sqrt{Var(N_c(t))}}.$$

The deposition of variance, $w_d(t)$ and $w_c(t)$ increases with t when t is large enough. This character is also consistent with the important intuition that ($N_d(t)$ or $N_c(t)$), which is observed for large t is more reliable to predict the future ($N_d(t)$ or $N_c(t)$). On the other hand, because of the increasing variance, if the WLS estimation method is applied with NHPP models, more weight will be addressed on the early data points, which will be opponent to the previous intuition, and deteriorate the prediction accuracy.

To obtain the minimum of S^2, a set of initial parameters is adopted to the mini-search function in MATLAB, and iterations are executed with different initial parameters decided by the last estimation.

2.4. Comparison Criteria

In order to make a comparison of various models' performance, mean square error (MSE) is used to quantify the difference between actual data and the estimated data. A smaller MSE represents a smaller fitting error and better performance. The MSE is generally defined as

$$\begin{cases} MSE_d(I_d) = \frac{1}{I_d} \sum_{k=1}^{I_d} \left(\frac{m_d\left(t_k^d\right) - n_k}{n_k} \right)^2 \\ MSE_c(I_c) = \frac{1}{I_c} \sum_{j=I_c+1}^{I_c} \left(\frac{m_c\left(t_j^c\right) - m_j}{m_j} \right)^2 \end{cases}, \tag{17}$$

Copyright © 2016 John Wiley & Sons, Ltd.

DOI: 10.1002/stvr

10991689, 2016, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/stvr.1600 by PUC/PR - Pontificia Universidade Catolica do Parana, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

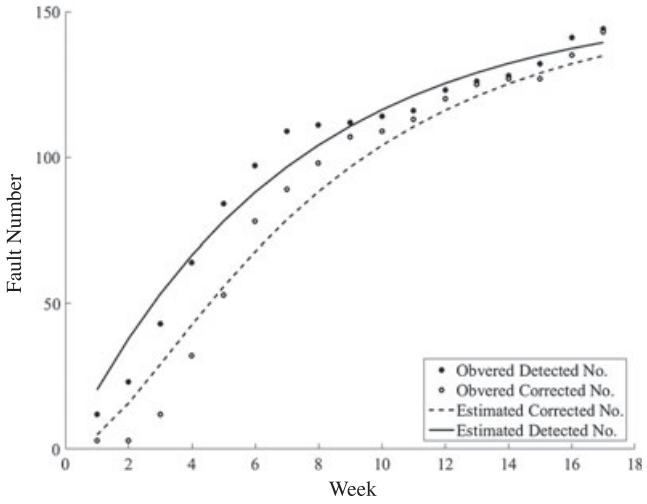


Figure 1. Actual and Estimate Fault Detection and Correction Processes.

here k is the size of the selected data set, b_i is the actual number of (detected or corrected) faults by time t_i, and $m(t_i)$ is the estimated number of (detected or corrected) faults by time t_i.

To study the prediction accuracy of the models, the variable-term prediction is used to quantify this ability [32]. This approach predicts the detected fault number $m_d(t_k)\{k = l_d + 1, l_d + 2, \ldots, N'\}$ and the corrected fault number $m_c(t_j)\{j = l_c + 1, l_c + 2, \ldots, M'\}$, using the partial data-set $\{m_d(t_k)\{i = 0, 1, 2, \ldots, l_d\}$ and $m_c(t_j)\{j = 0, 1, 2, \ldots, l_c\}$. The measure for these errors is mean of relative errors (MRE), which is

$$\begin{cases} MRE_d(l_d) = \frac{1}{N'-I_d+1} \sum\limits_{k=I_d+1}^{N'} \left| \frac{m_d(t_k^d)-n_k}{n_k} \right| \\ MRE_c(l_c) = \frac{1}{M'-I_c+1} \sum\limits_{j=I_c+1}^{M'} \left| \frac{m_c(t_j^c)-m_j}{m_j} \right| \end{cases}. \tag{18}$$

The MRE of FDP and FCP is calculated separately. The lower MRE indicates the better prediction accuracy.

3. SOME APPLICATION EXMAPLES

In this section, two application examples are shown to illustrate the approach for FDP and FCP models and their parameter estimation with WLS method. Some other existed models are compared to illustrate the WLS being an effective method. In the first part, an earlier dataset with equal time interval $\left(t_k^d = t_j^c, \text{ when } k = j\right)$ is used. In the second part, a new and real application datasets with unequal time interval $\left(t_k^d \neq t_j^c, \text{ when } k = j\right)$ is applied.

3.1. Application to an existing data set

In this part, the dataset published by *Xie* [19] is used as an example and shown in Table I. This dataset is grouped testing data with equal time interval in FDP and FCP. Assuming that both of the faults detecting time and correcting duration are exponentiallly distributed, the proposed modelling framwork are discribed as Equation (3) and Equation (12). Using WLS estimate, the estimated parameters are $\hat{N} = 152.8677, \hat{b} = 0.1429, \hat{\mu} = 0.5641$. The estimated FDP and FCP are shown in Figure 1. The Figure 2 illustrates that when $t > 6$ weeks, the weight of FDP and FCP is increasing with t, that is, more weight is assigned on the latest data errors.

3.1.1. Predictability comparison. To illustrate the predictability of the proposed model, three different types of models are compared in this part. These models are

Copyright © 2016 John Wiley & Sons, Ltd.

DOI: 10.1002/stvr

10991689, 2016, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/stvr.1600 by PUC/PR - Pontificia Universidade Catolica do Parana, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table I. Dataset published in [19].

Test Week	detected No.	corrected No.	Test Week	detected No.	corrected No.
1	12	3	10	114	109
2	23	3	11	116	113
3	43	12	12	123	120
4	64	32	13	126	125
5	84	53	14	128	127
6	97	78	15	132	127
7	109	89	16	141	135
8	111	98	17	144	143

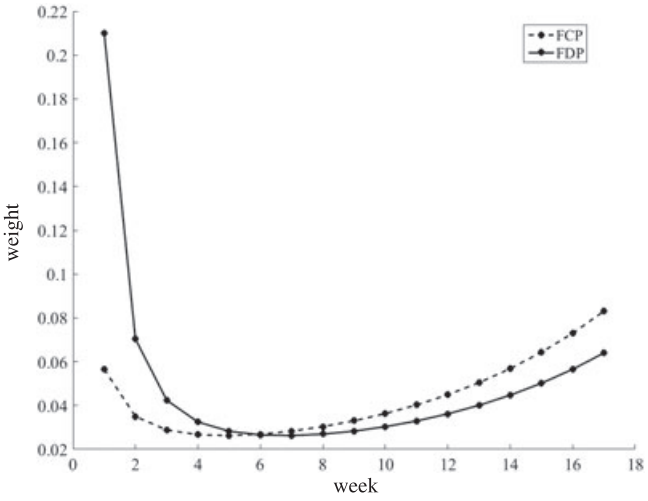


Figure 2. Weight of the proposed model.

Model 1, the GOS-based exponential time delay (ETD) model (proposed model) with WLS estimate.
Model 2, the ETD model with OLS estimate.
Model 3, the NHPP based ETD model with WLS estimate.

While the assumptions of fault detection process in the previous three models are quite different from each other, the assumptions of time delay for the fault correction is the same. Thus, the mean value function of each model is in similar format. To justify the assumptions of the proposed framework, those three models are compared.

For simplification, assume each fault detected distribution obeys the exponential distribution with the same fault detection rate b, and the exponential correction time delay with the correction rate μ.

Previous models are adopted in first 12 weeks to predict the remaining 5 weeks. The $MRE_d(12)$ and $MRE_c(12)$ are used as measures of prediction ability. The results of estimation and predictability of these models are listed in Table II and shown in Figure 3. The 95% confidence intervals for model 1 and model 3 are produced by bootstrap algorithm. It shows that the proposed model has the best overall prediction accuracy and achieve much better agreement between actual and estimated FDP. While, Model 3 did much worse in prediction than other two models, tending to be over-prediction on the faults numbers. The results also shows that the OLS estimate can lead to over fitting, which may influence the predictability.

Comparing the 95% confidence intervals, model 1 has the narrowest interval for N, which is the expected number of initial faults in software. The estimates of N in model 2 and model 3 are located near the upper bound of the estimate

3.1.2. Optimal Software Release Time Analysis. One of the most significant applications of software reliability models is the determination of software release time. Some cost models have been

Copyright © 2016 John Wiley & Sons, Ltd.

DOI: 10.1002/stvr

10991689, 2016, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/stvr.1600 by PUC/PR - Pontificia Universidade Catolica do Parana, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

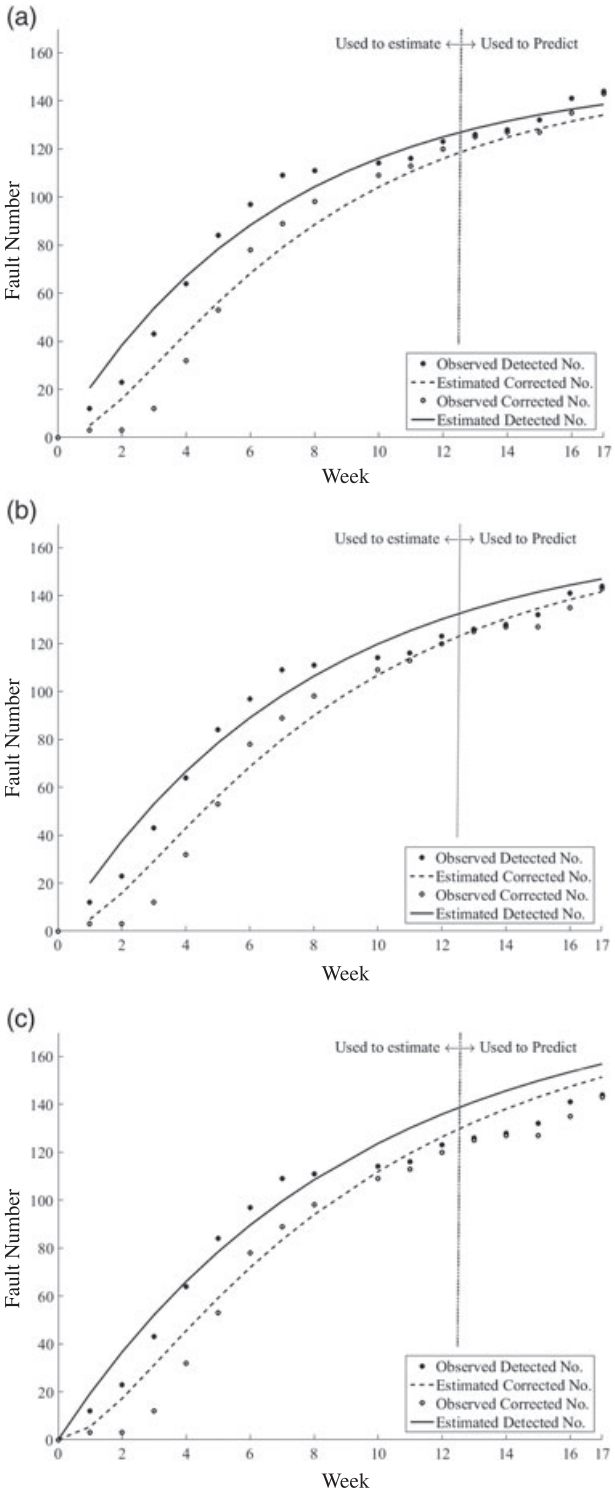


Figure 3. Actual and Estimate Fault Detection and Correction Processes.

proposed in previous works [33]. Considering with FCP, the cost model can be written as

$$C(T) = c_{1d} \times m_d(T) + c_{1c} \times m_c(T) + c_2 \times (N - m_c(T)) + c_3 \times T \tag{19}$$

where c_{1_d} and c_{1_c} are the expected cost of finding and removing a fault during test phase, since both of the works are carried on separately. c_2 is the cost for the per remaining uncorrected fault, c_3

Copyright © 2016 John Wiley & Sons, Ltd.

DOI: 10.1002/stvr

10991689, 2016, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/stvr.1600 by PUC/PR - Pontificia Universidade Catolica do Parana, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table II. Estimates and predictability.

Result		Model		
		Proposed model	Exponential time delay model	NHPP model with WLS
Parameter		$\hat{N} = 150.9730 \pm 17.4882$ $\hat{b} = 0.1465 \pm 0.0407$ $\hat{\mu} = 0.5677 \pm 0.2317$	$\hat{N} = 165.5349 \pm 40.1480$ $\hat{b} = 0.1286 \pm 0.0570$ $\hat{\mu} = 0.5777 \pm 0.2527$	$\theta = 165.4736 \pm 40.4885$ $\hat{b} = 0.1314 \pm 0.0748$ $\hat{\mu} = 0.3970 \pm 0.0193$
Prediciton	$MRE_d(12)$	0.027	0.053	0.060
	$MRE_c(12)$	0.030	0.025	0.020
	MRE_{all}	0.057	0.078	0.080

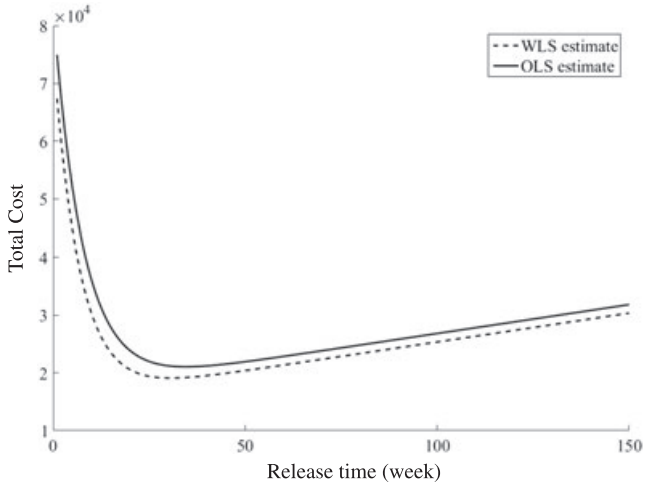


Figure 4. Estimation of total cost.

is the expected cost per unit time for testing. To conform the existence of optimal release time of T, c_2 must larger than c_{1c}, which means removing a remaining uncorrected fault after release will cost more than that during testing. Here, these cost items are assumed as

Figure 4 shows the plot of the total cost comparing with the proposed model with WLS estimate and ETD model with OLS estimate. The optimal release times for both of the models are, 31 with a total cost of 19090 and 35 with a total cost of 21052, respectively. Because the WLS estimates has a better prediction performance (shown earlier), its optimal release time is more accuracy than OLS. Also, because of its over fitting phenomenon, the OLS estimation can be a reference to a conservative estimation of software release time, which means the cost is more likely to be bigger than real situation.

3.2. Application to a new and real data set with unequal time interval

To illustrate more about the proposed model and WLS estimate, an original dataset collected in a real software project is applied in this part. In the software development of instrument and control systems for a nuclear power plant, some fault detection and correction datasets are collected. During the testing process, there are actually two groups participating at the same time. One debugging group detects the faults in current software via Auto Test [34] and submits them to the developing group who is assigned to correct faults. Thus, the collected data include the fault submitted time, which is used as fault detection data, fault corrected time, which is used as fault correction data, and some additional properties such as fault severity, priority of correction process. A dataset named as Sys. A2 is chosen as the research object for this paper.

The normalized cumulative faults number of Sys. A2 are listed in Tables III and IV. The testing days are counted with calendar time from which holidays and weekends are eliminated. Because

Copyright © 2016 John Wiley & Sons, Ltd.

DOI: 10.1002/stvr

10991689, 2016, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/stvr.1600 by PUC/PR - Pontificia Universidade Catolica do Parana, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table III. Cumulative Detected Fault number of Sys.A2.

Test day	detected No.	Test day	detected No.	Test day	detected No.
1	23	17	640	34	898
2	96	20	672	36	904
3	163	21	703	37	916
4	250	22	706	42	919
5	282	23	730	43	924
6	343	24	747	44	939
7	375	25	756	45	948
8	398	26	785	46	956
9	424	27	797	52	971
10	445	28	828	61	983
11	459	29	866	63	988
12	465	30	869	64	991
13	541	31	875	65	994
14	593	32	892	86	997
15	637	33	895	87	1000

Table IV. Cumulative Corrected Fault number of Sys.A2.

Test day	Corrected No.	Test day	Corrected No.	Test day	Corrected No.
1	6	18	523	36	811
2	15	19	529	38	837
3	17	23	532	39	849
4	20	24	535	40	855
5	23	25	541	41	860
6	35	26	581	42	863
7	44	27	608	43	869
10	55	28	622	44	884
11	84	29	631	45	927
12	105	30	660	46	942
13	128	31	689	65	945
14	163	32	712	68	959
15	203	33	744	69	962
16	302	34	773	70	965
17	483	35	794	74	968

the submission time is not equal to the fault detected time and maybe delayed for random days, the sequence of test days is not continuous.

3.2.1. Performance comparison. Because of the bad performance of Model 3, this part focuses on the comparison of Model 1 and Model 2 with previous datasets.

The first 40 days' test data of Sys.A2 is used to estimate the model parameters. Figure 5 shows the Prediction curves with both of the OLS estimate (Model 2) and WLS estimate (Model 1). The estimated parameters and the fit-goodness are shown in Table V. The result demonstrates that the OLS is better than WLS in fitting observed data set, but WLS has a better accuracy in prediction.

In Table V, the upper bound of N with WLSE is 1034.7626, which is lower than that of the LSE of N. Thus, in this case, the WLSE of N is significantly different from the LSE.

Figure 6 presents the weights of WLS, both the weights of FDP and FCP begin to increase with t when $t_d \geq 11$ and $t_c \geq 17$.

The variable-term prediction results of WLS compared with OLS for FDP and FCP are shown in Figure 7, respectively. A series of MRE from MRE(20) to MRE(70) are plotted against the end time of data used to estimate the model. Results illustrate that the WLS method has much lower prediction errors than the OLS method in both FDP and FCP, and the WLS method presents a good prediction in FDP with early data points, which is critical for making management scheme, such as prediction of release time, arrangement of testing effort.

Copyright © 2016 John Wiley & Sons, Ltd.

DOI: 10.1002/stvr

10991689, 2016, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/stvr.1600 by PUC/PR - Pontificia Universidade Catolica do Parana, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

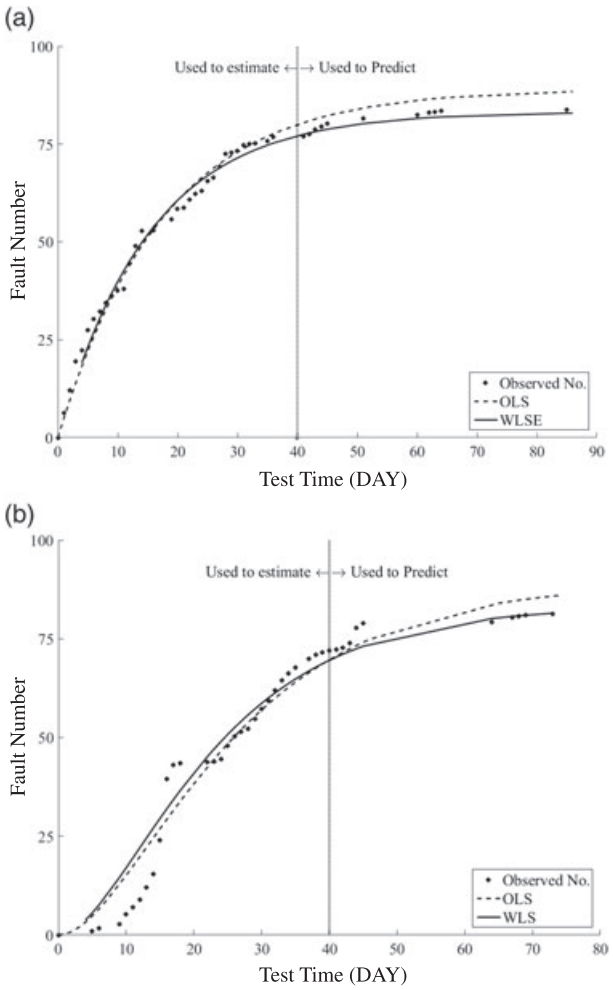


Figure 5. Fault Detection and Correction with Exponential Time Delay with OLS and WLS estimate (a) Detection (b) Correction.

Table V. Fitted Results with Exponential Time Delay.

		Estimates	
Result		Proposed model	Exponential time delay model
Parameter	N	967.63 ± 79.3450	1035.8424 ± 103.7698
	b	0.0652 ± 0.0116	0.0570 ± 0.0093
	μ	0.1091 ±0.0359	0.0974 ± 0.0121
FDP	MSE of Fitness	766.7523	924.5411
	MRE of Prediction	0.0291	0.1744
FCP	MSE of Fitness	5757.2243	4838.2014
	MRE of Prediction	0.0756	0.1424

3.2.2. Optimal software release time analysis. Figure 8 shows the plot of the total cost with the previous conditions, with comparison of different estimations. The optimal release times for the OLS estimation and WLS estimation model are 76 with a total cost of 45088 and 70 with a total cost of 43204, respectively. Considering the cost and to ensure high quality of the software, software managers should adopt WLS estimates as it is more stable, and has better prediction performance

Copyright © 2016 John Wiley & Sons, Ltd.

DOI: 10.1002/stvr

10991689, 2016, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/stvr.1600 by PUC/PR - Pontificia Universidade Catolica do Parana, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

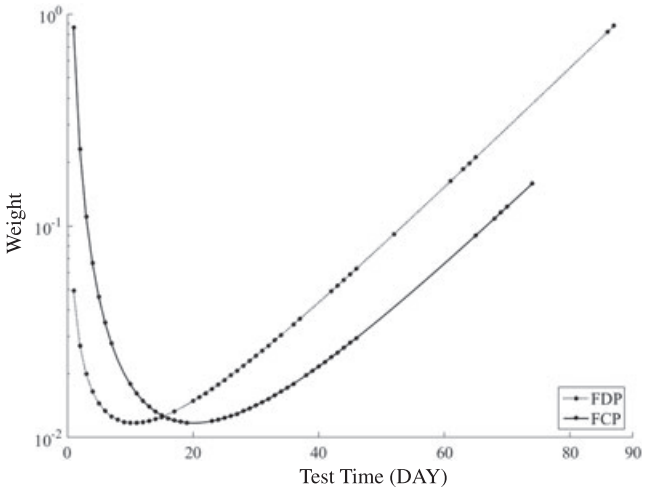


Figure 6. Weight of WLS estimation.

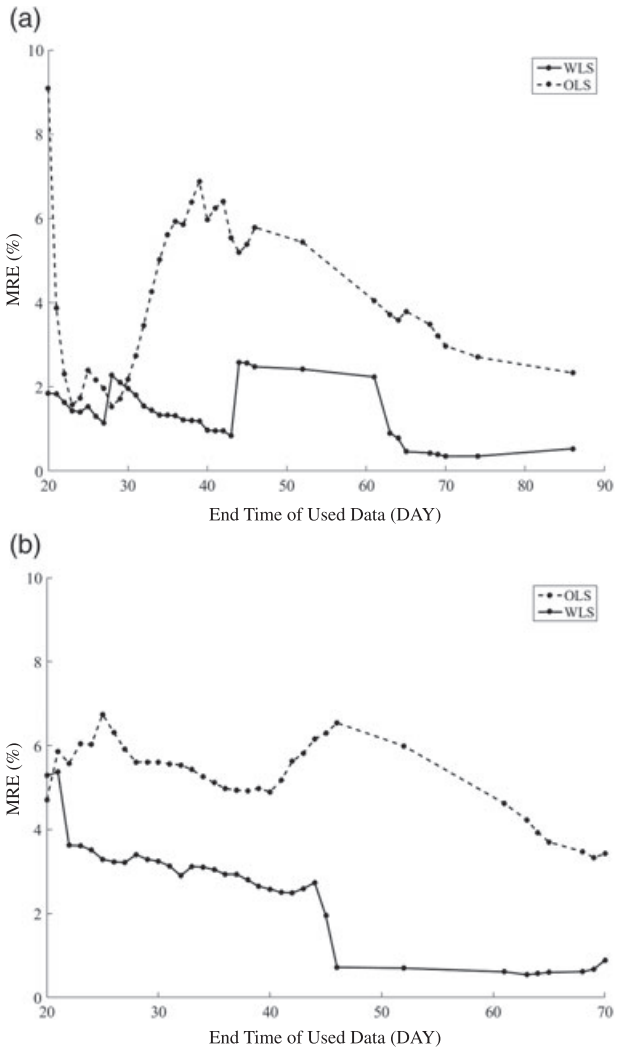


Figure 7. variable-term prediction ability of FDP (a) and FCP (b).

Copyright © 2016 John Wiley & Sons, Ltd.

DOI: 10.1002/stvr

10991689, 2016, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/stvr.1600 by PUC/PR - Pontificia Universidade Catolica do Parana, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

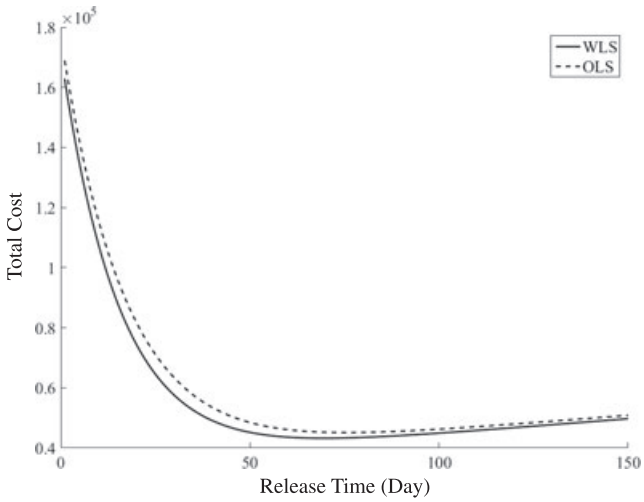


Figure 8. Estimation of total cost.

(shown earlier). Also, the OLS estimation can be a reference to conservative estimates of software release time. These results are consistent with the former example.

4. CONCLUSION

In this paper, a novel modeling framework for software FDP and FCP is developed and its parameter estimation with WLS method is studied. The research is based on the hypothesis on GOS that the detection time and correction duration of each fault approximately and independently follows certain stochastic distribution, and the developed FDP and FCP models have a general framework. Using two datasets of which one is from published work and the other one is collected during the development of a practical software system, the application of the models is illustrated that the proposed model with the WLS estimate has lower prediction errors. This is because the proposed model contains the fault correction information as well as the detection information, and the WLS estimation weights more on the errors of recent data, which have more insights with the future test process.

There are some limitations in current research, such as it sets the exponential order statistic as example, and this model fails to fit the S-shaped fault-detection dataset, which would be improved in future research. Moreover, other data sets should be applied and analyzed to support the research conclusions, and more data information like severity, change-point, should be considered to amend the software models.

ACKNOWLEDGEMENTS

This work has been partly supported by the grant from the National S&T Major Project (Grant No. ZX06901) and National Center for Mathematics and Interdisciplinary Sciences of Chinese Academy of Sciences. The authors are grateful to Professor Min Xie of Department of Systems Engineering and Engineering Management of City University of Hong Kong, for his encouragement and suggestions on this work.

REFERENCES

1. Lyu MR. Software reliability engineering: A roadmap. In *in: Proceedings of the Twenty-Ninth International Conference on Software Engineering*. Future of Software Engineering: Minneapolis, 2007; 153–170.
2. Xie M. Software reliability models—A selected annotated bibliography. *Software Testing, Verification & Reliability* 1993; **3**(1):3–28.
3. Dyer M. Software reliability — what is it? *Software Testing, Verification and Reliability* 1991; **1**:31–38.
4. Goel ALAL. Software Reliability Models: Assumptions, Limitations, and Applicability. *IEEE Transactions on Software Engineering* 1985; **SE-11**:1411–1423.

Copyright © 2016 John Wiley & Sons, Ltd.

DOI: 10.1002/stvr

10991689, 2016, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/stvr.1600 by PUC/PR - Pontificia Universidade Catolica do Parana, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

5. Lyu MR. *Handbook of Software Reliability Engineering*. IEEE computer society press: CA, 1996.
6. Okamura H, Dohi T, Osaki S. Software reliability growth models with normal failure time distributions. *Reliability Engineering & System Safety* 2013; **116**:135–141.
7. Yamada S, Fujiwara T. Testing-domain dependent software reliability growth models and their comparisons of goodness-of-fit. *International Journal of Reliability Quality and Safety Engineering* 2001; **8**:205–218.
8. Aktekin T, Caglar T. Imperfect debugging in software reliability: A Bayesian approach. *European Journal of Operational Research* 2013; **227**:112–121.
9. Huang CY, Kuo CS, Luan SP. Evaluation and Application of Bounded Generalized Pareto Analysis to Fault Distributions in Open Source Software. *IEEE Transactions on Reliability* 2014; **63**:309–319.
10. Jelinski Z, Moranda PB. Statistical computer performance evaluation. *Software Reliability Research* 1972:465–484.
11. Miller DR. Exponential order statistic models of software reliability growth. *IEEE Transactions on Software Engineering* 1986; **SE-12**(1):12–24.
12. Ross SM. Statistical Estimation of Software Reliability. *IEEE Transactions on Software Engineering* 1985; **SE-11**:479–483.
13. Kapur PK, Pham H, Anand S, Yadav K. A unified approach for developing software reliability growth models in the presence of imperfect debugging and error generation. *IEEE Transactions on Reliability* 2011; **60**:331–340.
14. Okamura H, Dohi T. Unification of Software Reliability Models Using Markovian Arrival Processes. In *2011 IEEE 17th Pacific Rim International Symposium on Dependable Computing (PRDC)*. IEEE: 345 E 47TH ST, NEW YORK, NY 10017 USA, 2011; 20–27.
15. Koh SLP. Weighted Least-Square Esimate for Software Error Intensity. *Journal of the Chinese Institute of Industrial Engineers* 2008; **25**(2):162–173.
16. Gokhale SS, Lyu MR, Trivedi KS. Analysis of Software Fault Removal Policies Using a Non-Homogeneous Continuous Time Markov Chain. *Software Quality Journal* 2004; **12**:211–230.
17. Schneidewind NF. Modelling the fault correction process. *in Proceedings of the 12th International Symposium on Software Reliability Engineering.* IEEE Computer Society Press: Los Alamitos, CA, 2001; 185–190.
18. Wu YP, Hu QP, Xie M, Ng SH. Modeling and Analysis of Software Fault Detection and Correction Process by Considering Time Dependency. *IEEE Transactions on Reliability* 2007; **56**:629–642.
19. Xie M, Hu Q, Wu Y, Ng S. A study of the modeling and analysis of software fault - detection and fault - correction processes. *Quality and Reliability Engineering International* 2007; **56**:459–470.
20. Shibata K, Rinsaka K, Dohi T, Okamura H. *Quantifying Software Maintainability Based on a Fault-Detection/Correction Model 13th Pacific Rim International Symposium on Dependable Computing (PRDC 2007)*. IEEE: Melbourne, 2007.
21. Huang CY, Hung TY. Software reliability analysis and assessment using queueing models with multiple change-points. *Computers & Mathematics with Applications* 2010; **60**:2015–2030.
22. Hu QP, Xie M, Ng SH, Levitin G. Robust recurrent neural network modeling for software fault detection and correction prediction. *Reliability Engineering & System Safety* 2007; **92**:332–340.
23. Peng R, Li YF, Zhang WJ, Hu QP. Testing effort dependent software reliability model for imperfect debugging process considering both detection and correction. *Reliability Engineering & System Safety* 2014; **126**:37–43.
24. Sahinoglu M, Can Ü. Alternative Parameter Estimation Methods for the Compound Poisson Software Reliability Model with Clustered Failure Data. *Software Testing, Verification & Reliability* 1997; **7**(1):35–57.
25. Ishii T, Dohi T, Okamura H. Software Reliability Prediction Based on Least Squares Estimation. *Quality Technology and Quantitative Management* 2012; **9**:243–264.
26. Jukic D. The L-p-norm estimation of the parameters for the Jelinski-Moranda model in software reliability. *International Journal of Computer Mathematics* 2012; **89**(4):467–481.
27. Draper NR, Smith H. *Applied Regression Analysis*, 3rd ed. Wiley series in probability and statistics: New York, 1998.
28. Ohishi K, Okamura H, Dohi T. Gompertz software reliability model: Estimation algorithm and empirical validation. *Journal of Systems and Software* 2009; **82**:535–543.
29. Huang CY, Lyu MR, Kuo SY. A unified scheme of some Nonhomogenous Poisson process models for software reliability estimation. *IEEE Transactions on Software Engineering* 2003; **29**:261–269.
30. Musa JD, Iannino A, Okumoto K. *Software Reliability: Measurement, Prediction, Application*. McGraw-Hill, Inc.: New York, 1987.
31. Langberg N, Singpurwalla ND. A unification of some software-reliability models. *Siam Journal on Scientific and Statistical Computing* 1985; **6**:781–790.
32. Malaiya YK, Karunanithi N, Verma P. Predictability of software-reliability models. *IEEE Transactions on Reliability* 1992; **41**(4):539–546.
33. Chiu KC, Ho JW, Huang YS. Bayesian updating of optimal release time for software systems. *Software Quality Journal* 2009; **17**:99–120.
34. Ciupa I, Pretschner A, Oriol M, Leitner A, Meyer B. On the number and nature of faults found by random testing. *Software Testing Verification & Reliability* 2011; **21**(1):3–28.

Copyright © 2016 John Wiley & Sons, Ltd.

DOI: 10.1002/stvr

10991689, 2016, 5, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/stvr.1600 by PUC/PR - Pontificia Universidade Catolica do Parana, Wiley Online Library on [17/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License